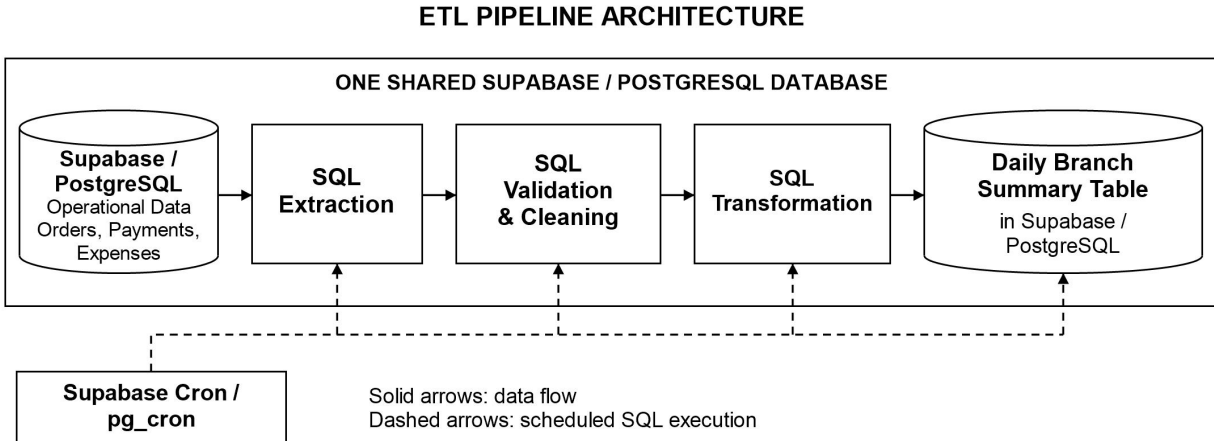


1. Pipeline Overview

The proposed I&C Laundry Daily Branch Reporting ETL Pipeline will extract operational records from the shared Supabase PostgreSQL database used by the Main, Calzada, and Nasugbu branches. Its purpose is to prepare consistent information for sales monitoring, payment collection reporting, laundry-demand analysis, branch comparisons, and daily customer activity. The design retains the previously selected stack: PostgreSQL / Supabase, Supabase Cron / pg_cron, SQL, and Reporting Summary Tables.

The pipeline will validate and standardize committed records, calculate daily measures, and load one reporting table in the same database. Order value will be reported on the order creation date, cash received on the payment collection date, and expenses on the recorded expense date. These measures will support historical dashboards and reports and can provide inputs for forecasting; forecast generation itself is outside this pipeline. Source definitions follow the database schema supplied by the project owner. The scheduled ETL function, staging tables, reporting table, and application integration remain proposed, not confirmed as deployed.

2. Pipeline Architecture



All processing and storage remain in PostgreSQL. Solid arrows show data movement between stages; dashed arrows show scheduled orchestration of the SQL refresh. The reporting table would be read through the application's authorized reporting layer.

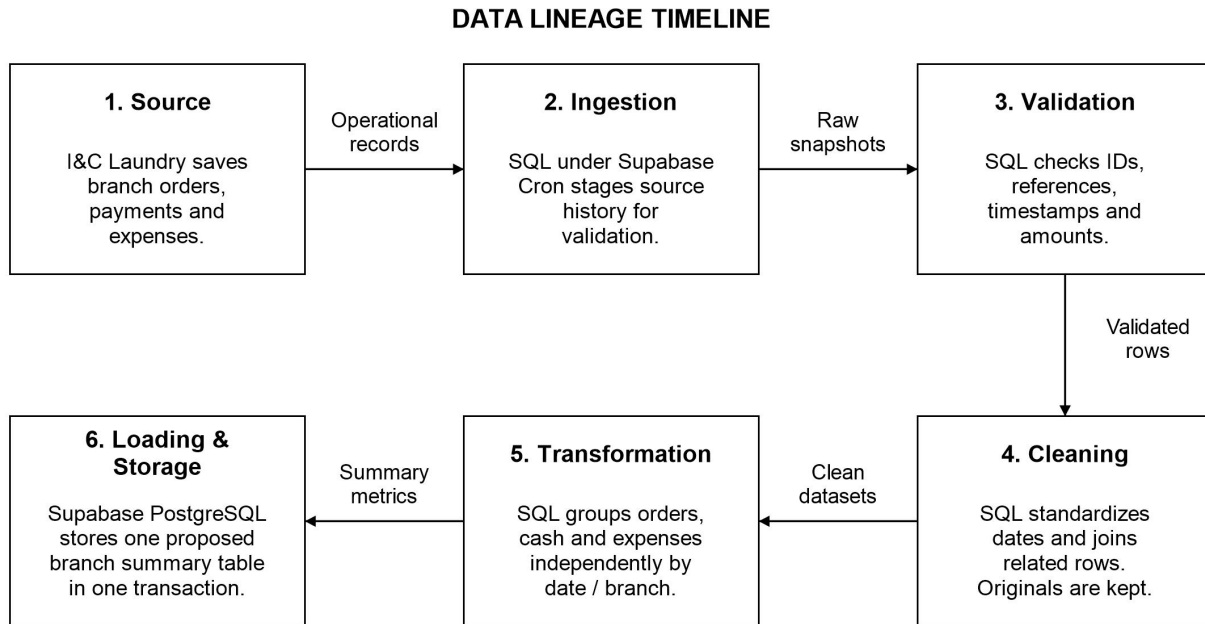
3. Pipeline Metadata

Metadata	Description
Pipeline Name	I&C Laundry Daily Branch Reporting ETL Pipeline
Purpose	Extract, validate, clean, transform, and load daily measures for branch operations, laundry demand, order value, received payments, expenses, and customer activity.
Technologies / Tools Used	PostgreSQL / Supabase; Supabase Cron / pg_cron; SQL; Reporting Summary Tables.
Data Source	Five tables from the supplied schema: public.orders, public.payments, and public.expenses, with public.branches and public.customers as references. Use branch IDs for grouping and branches.branch_name for labels.
Destination / Storage	One proposed analytics.daily_branch_summary table in the existing Supabase PostgreSQL database.
Ingestion Tool	PostgreSQL SQL SELECT statements copy source records into transaction-local staging tables. No external ingestion server is introduced.
Transformation Tool	SQL performs validation, timezone conversion, joins, filtering, independent aggregation, and reconciliation before publishing results.
Orchestration Tool	Supabase Cron / pg_cron invokes the proposed analytics.refresh_daily_reports() function. Job name: ic_laundry_daily_reports.
Data Storage	Operational records stay in public tables. The proposed reporting table is stored in analytics. Temporary staging data exists only during the refresh transaction.
Schedule	Daily at 01:00 Asia/Manila (UTC+08:00). With the scheduler configured for UTC, the cron expression is 0 17 * * *. Process business dates earlier than the local date at run start.

Metadata	Description
Dependency	Source tables matching the supplied schema; valid branch and order references; missing required reporting dates or branch assignments resolved. Verify pg_cron, the proposed function and target, and authorized source-read/target-write permissions before deployment.
Configurations	PHP currency; Asia/Manila business dates; exact NUMERIC arithmetic; one consistent source snapshot and run timestamp; transaction-scoped advisory lock to prevent overlapping runs. Rebuild completed-day history to include corrections and late collections. A failed validation rolls back publication; inspect job history before rerunning.
Connections Between Technologies	Supabase Cron invokes SQL in PostgreSQL. SQL reads the shared operational tables, stages and groups records, then writes the branch reporting table in one transaction. Authorized application queries consume the published summaries.
Pipeline Steps	1. Source operational records. 2. Ingest a consistent snapshot. 3. Validate keys, dates, and amounts. 4. Clean and standardize staged data. 5. Transform into daily metrics. 6. Load and store reconciled summaries.

4. Data Lineage

The diagram follows one scheduled ETL run. Its six stages describe the sequence of transformations, not their execution duration. Source records are preserved; cleaning takes place in the staged copy.



Stage	Input	Process	Output
Source	I&C Laundry operations across three branches.	The application records orders, payment-ledger entries, expenses, and reference data in the shared Supabase PostgreSQL database.	Committed operational records.
Ingestion	Source tables and related branch and customer IDs.	Cron invokes SQL. Capture one transaction snapshot and run timestamp; copy relevant history into temporary stages. Retain invalid or missing dates for validation.	Raw staged records at their original row grain.
Data Validation	Staged orders, payments, and expenses.	Check IDs, dates, branches, order links, allowed statuses, and amounts. Nullable source branches and order dates must resolve before publishing. Invalid records stop the refresh.	Validated source records.

Stage	Input	Process	Output
Cleaning	Validated staged records.	Derive local dates from orders.created_at and payments.paid_at; retain expenses.expense_date. Join branches by ID and use branch_name for labels.	Consistently dated and related datasets.
Transformation	Clean datasets through yesterday.	Aggregate orders, cash collections, and expenses independently by their own dates and branch. Merge aggregate keys without multiplying detail rows.	Daily branch metrics.
Loading and Storage	Reconciled aggregate datasets.	Check totals against each staged source using the same dates and filters. Replace the target table atomically and set refreshed_at. Roll back on failure; repeat runs do not append duplicates.	Published daily branch summary table.

5. Schema Metadata

Source definitions follow the owner-supplied 21-table schema, not a direct database inspection. This table lists the five sources and selected columns used by the ETL, followed by one proposed target; it is not a full application data dictionary. PK means primary key; FK means foreign key. “NULL” marks nullable columns; other listed columns are NOT NULL. Source NUMERIC fields have no declared precision or scale.

Database Table	Columns and Data Types	Primary Key	Foreign Keys and Relationships
public.branches (existing source)	id — UUID branch_name — TEXT name — TEXT (NULL) code — TEXT (NULL)	id	One branch has many orders, payments, expenses, and reporting rows. branch_name supplies the label; name and code are optional.
public.customers (existing reference)	id — UUID	id	Referenced by orders.customer_id. A customer can have many orders. Contact details are not extracted.
public.orders (existing source)	id — UUID branch_id — UUID (NULL) customer_id — UUID (NULL) created_at — TIMESTAMPTZ (NULL) status — TEXT weight_kg — NUMERIC total_price — NUMERIC	id	branch_id → branches.id customer_id → customers.id One order can have many payment entries.
public.payments (existing ledger)	id — UUID order_id — UUID branch_id — UUID (NULL) amount — NUMERIC paid_at — TIMESTAMPTZ source — TEXT	id	order_id → orders.id branch_id → branches.id Each collection is a separate ledger row linked to its order and branch.
public.expenses (existing source)	id — UUID branch_id — UUID (NULL) amount — NUMERIC expense_date — DATE deleted_at — TIMESTAMPTZ (NULL)	id	branch_id → branches.id Include only non-deleted expenses. A missing branch must be resolved before publication.

Database Table	Columns and Data Types	Primary Key	Foreign Keys and Relationships
analytics. daily_branch_ summary (proposed target)	report_date — DATE branch_id — UUID order_count — BIGINT released_order_count — BIGINT unique_customer_count — BIGINT total_weight_kg — NUMERIC(14,2) order_value_php — NUMERIC(14,2) payment_count — BIGINT cash_received_php — NUMERIC(14,2) expense_total_php — NUMERIC(14,2) refreshed_at — TIMESTAMPTZ	Composite: report_date, branch_id	branch_id → public.branches.id One row per date and branch. Merge order, payment, and expense aggregates using the union of their keys.

Schema interpretation and ETL rules

Source versus ETL requirements. The source permits NULL branch IDs in orders, payments, and expenses, and NULL orders.created_at. The proposed refresh requires resolved branches and dates before publication; it must not silently discard these rows or invent values. This reporting rule does not change the source constraints.

Staging: proposed pg_temp.etl_orders_stage, pg_temp.etl_payments_stage, and pg_temp.etl_expenses_stage retain their source IDs and selected fields and add report_date DATE. They exist only for one refresh and are dropped at transaction end.

Metric rules. Order counts, weight, and order value exclude cancelled orders and use the local creation date. Released-order counts describe current status within that creation-date group, not daily releases. Customer counts use distinct non-null IDs within a date and branch; do not sum them to infer distinct customers over a wider period.

Cash rules. cash_received_php is SUM(payments.amount), dated by paid_at, with one count per ledger row. Later collections belong to the later date, not the order date. Do not also sum orders.amount_paid. This proposed metric retains recorded collections after cancellation, which is not evidence of a refund. It is gross cash, not net income; net collections require an approved refund ledger. Legacy backfill timestamps are best-available estimates, not exact historical collection times.

Grouping and integrity. Branch summary keys include dates with only payments or expenses. Missing measures are zero. All target columns are required, and the composite primary key is (report_date, branch_id). Enforce unique keys, nonnegative totals, and valid branch references. Reconcile each metric against its staged source using the same dates and filters. Any discrepancy aborts the refresh.

Source definitions. The five source tables and the selected column types, nullability, and relationships are based on the supplied database schema. Only fields required for this reporting pipeline are documented; this is not a complete application schema. Cash received is derived from payments.amount and payments.paid_at, not the order payment summary.

Scope and limitations. This pipeline covers completed-day, branch-level reporting for orders, payment collections, expenses, and customer activity across the three branches. It uses orders, payments, expenses, branches, and customer identifiers to prepare analytics.daily_branch_summary. Live transaction processing, inventory, loyalty, audit monitoring, authentication, and forecast generation remain outside its scope. Customer contact details and credentials are not extracted. The scheduled refresh and summary table are proposed components, not verified deployments.

6. Technology Justification

PostgreSQL / Supabase

The system already uses Supabase PostgreSQL for shared operational data. Keeping extraction and reporting storage in the same database fits the existing relational structure and avoids an additional data-transfer component. Keys support consistent relationships, and a transactional publication step can preserve the last successful summaries when a refresh fails. The job should run off-peak and its runtime should be monitored so analytical processing does not disrupt branch transactions.

Supabase Cron / pg_cron

This is the selected scheduler for the proposed daily SQL refresh. It provides an in-database trigger for the pipeline and job history for operational review, without introducing Apache Airflow or another scheduling platform. The configured scheduler timezone must be checked before enabling the 01:00 Asia/Manila schedule. An advisory lock in the refresh function prevents simultaneous runs from publishing conflicting results.

SQL

SQL is selected for ingestion, validation, cleaning, transformation, and loading because the required data is relational. The same proposed function can apply date rules, validate references, aggregate separate transaction streams, and reconcile final totals. Independent aggregation is important: directly joining orders to multiple payments and expenses could multiply rows and overstate the results. Centralizing these rules also makes the proposed calculations easier to review and test.

Reporting Summary Tables

This is the selected reporting storage approach, not an additional software product. The proposed analytics.daily_branch_summary table stores reusable measures at the date-and-branch level. It reduces repeated aggregation and separates order value from collected cash. Reports would display refreshed_at and identify the completed-day cutoff; live order tracking would continue to use operational records. This separation supports consistent historical reporting without presenting scheduled summaries as real-time transaction data.